

COMP7710

Programming Fundamentals

Today's lecture

Computational Thinking & Java Overview

Course Convenor
Bernardo Pereira Nunes



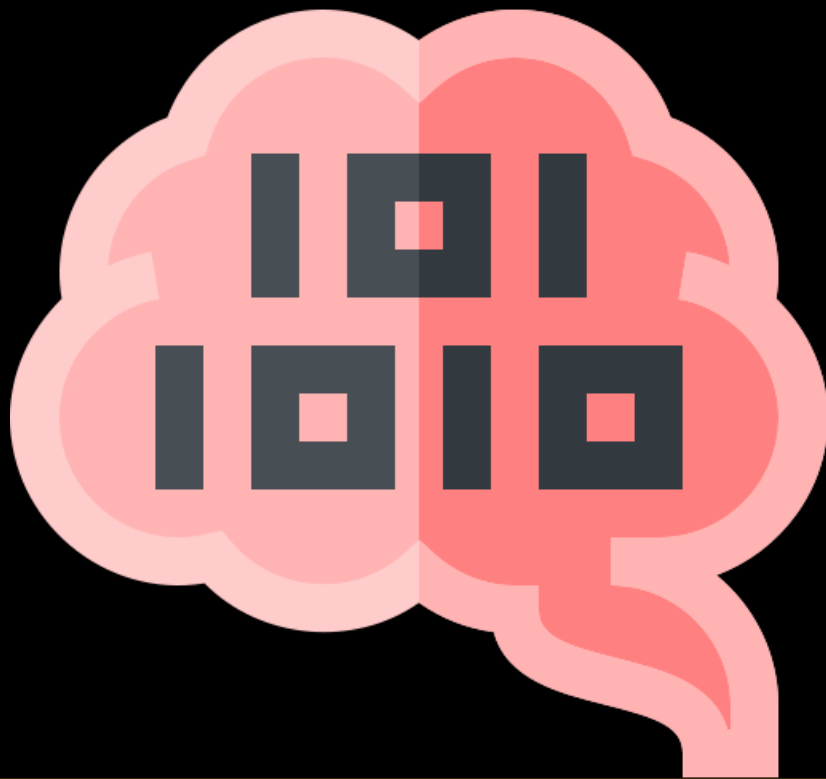
Australian
National
University

Agenda

- > What is Computational Thinking?
- > Code Examples
- > What is a Programming Language?
- > GenAI, Vibe Coding
- > Java Overview
- > Exercises



COMPUTATIONAL THINKING



WHAT IS COMPUTATIONAL THINKING?

Computational thinking is a problem-solving approach that draws on concepts and techniques from computer science to address complex problems and systems.



GUESS MY NUMBER

Objective

Guess a number between 0 and 255

Rule

You can only ask YES/NO questions



GUESS MY NUMBER

Potential Strategies

Random Guessing

Linear search

Binary Search



GUESS MY NUMBER

Potential Strategies

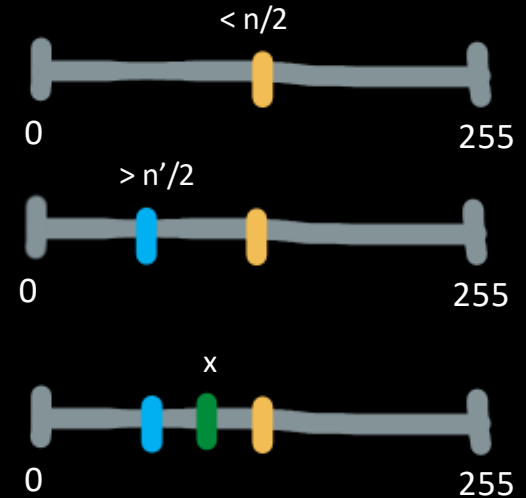
Random Guessing



Linear search



Binary Search



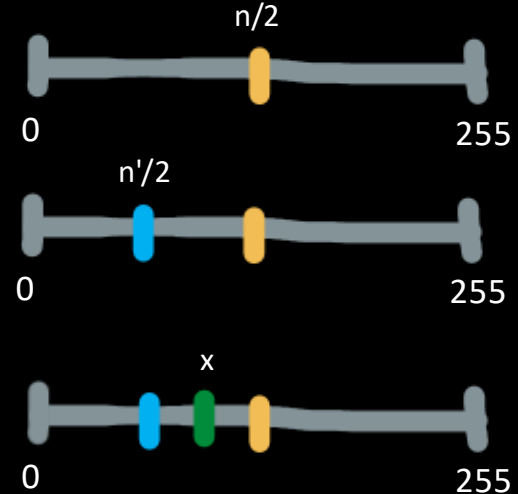
GUESS MY NUMBER

Pseudocode

1. Given a range of numbers and a number to be guessed
2. Pick the number in the middle of the range
3. If the number equals the number to be guessed
 - . you win a lollipop 🍭
 - . stop
4. Else if the number is greater than the number to be guessed
 - . new range is the left half
 - . go to Step 2
5. Else
 - . new range is the right half
 - . go to Step 2

*within the range
if the range
is not empty*

Binary Search



Do we all agree this algorithm be applied to problems other
than guessing numbers?



<CODING TIME>

L01_NumberGuessing



What is programming?

- > Programming is the act of **telling a computer exactly what to do**, step by step, to solve a problem
- > Computers do not “understand” problems... they execute instructions, in order, and without “human” interpretation.
- > Computers are fast, but **we are smart!**
- >> Within practical limits, **computers are as fast as we are smart in structuring the problem.**
- >> Computers read instructions, execute them as is, and can do that million of times per second.
- > Instructions are written in a formal language (with the goal of reducing ambiguity).



WHAT HAVE WE LEARNED SO FAR?

Conditions

- > A statement or expression that evaluates to either true or false
- > Control the flow of the program
- > Control the behavior of your programs based on different situations

Loops

- > Loops are used to automate repetitive tasks
- > They help reduce code duplication and make programs more efficient and flexible

Algorithms

- > A step-by-step set of instructions that, when followed, accomplishes a specific task



What is a programming language?

Programming languages are formal languages designed for expressing computation [1].

> A programming language is how we (humans) communicate with computers.

>> It can be seen as the **bridge** between *humans* and *computers*

(abstract ideas) *(precise instructions)*

>> There are a number programming languages (JAVA, C, C++, Python, GO, Scala, PHP, ...)

>>> used to build software in different application domains

>> low-level languages

vs

high-level languages

(little or no abstraction from machine instructions; used to develop OS, Embedded Systems)

COMP6300 covers this topic in more depth.

(abstracts hardware details; detached from machine details (architectural agnostic))

[1] Keith D. Cooper, Linda Torczon, Overview of Compilation, Editor(s): Keith D. Cooper, Linda Torczon, Engineering a Compiler (Third Edition), Morgan Kaufmann, 2023, Pages 1-26.



What is a programming language?

- > **Computer programs** are usually written in high-level languages
- > High-level languages are **designed to be readable and understandable by humans**
- >> They use words and structures similar to English (while, if, return)
- > Computers only understand (execute) machine code, which is represented in binary (0,1)
- > A program written in a high-level language is called **source code**
- >> Source code must be **translated into machine code**

 **compilers or interpreters**

...converts high-level code into machine code the computer can execute.

**note that any language can be compiled or interpreted.*



What is a programming language?

A language is compiled if execution is preceded by a translation step that

- (1) fully analyzes both the structure (**syntax**) and meaning (**semantics**) of the program, and*
- (2) produces an equivalent program in a significantly different form. [2]*

Compilers are generally structured as a series of phases:

> The first few phases (scanning, parsing, and semantic analysis) serve to analyse the source program. The final few phases (intermediate code generation, code improvement, and target code generation) serve to build a target program—preferably a fast one—whose semantics match those of the source. [2]

> A compiler translates the whole program into machine code at once, whereas an interpreter executes it statement by statement without prior compilation.

Note that this is a simplified explanation;
COMP8712 covers this topic in more depth.

[2] Michael L. Scott, Programming Language Pragmatics (Third Edition), Morgan Kaufmann, 2009, Pages 5-39, <https://doi.org/10.1016/B978-0-12-374514-9.00010-0>.



Will programming languages be impacted by GenAI?

Nah, yeah!

BIRD (Blg Bench for LaRge-scale Database Grounded Text-to-SQL Evaluation) is an industry standard for testing text-to-SQL solutions.

In the fast-evolving world of agentic development, natural language is becoming the standard for interaction. This shift is deeply connected to the power of operational databases, where a more accurate text-to-SQL capability is a major catalyst for building better, more capable agents. From empowering non-technical users to self-serve data, to accelerating analyst productivity, the ability to accurately translate natural language questions into SQL is a game-changer. As end-user engagements increasingly happen over chat, conversations become the fundamental connection between businesses and their customers.

In an earlier post, "[Getting AI to write good SQL: Text-to-SQL techniques explained](#)," we explored the core *challenges* of text-to-SQL — handling complex business context, ambiguous user intent, and subtle SQL dialects — and the general *techniques* used to solve them.

Today, we're moving from theory to practice. We're excited to share that Google Cloud has scored a new state-of-the-art result **on the BIRD benchmark's Single Trained Model Track**. We scored 76.13, ahead of any other single-model solution (higher is better). In general, the closer you get to the benchmark of human performance (92.96), the harder it is to score incremental gains.

COMP6240 may cover this topic in more depth.

Leaderboard - Execution Accuracy (EX)						
	Model	Code	Size	Self Consistency	Dev (%)	Test (%)
	Human Performance <i>Data Engineers + DB Students</i>					92.96
1 Oct 16, 2025	Gemini-SQL (Multitask SFT + Gemini-2.5-Pro) <i>Google Cloud Databases</i>		UNK	Few	72.62	76.13
2 July 08, 2025	Databricks RLVR 32B <i>Databricks Mosaic Research</i> <i>Databricks Mosaic Research</i>		32B	Few	-	75.68
3 Oct 30, 2025	DorySQL-3B-MOE <i>Amazon Ads-SpektrBot</i> <i>[Ravi Shankar]</i>		3B-MOE	Few	72.10	74.85
4 July 22, 2025	Sophon-Text2SQL-32B <i>ByteDance DataPlatform-LLM</i>		32B	Many	72.43	74.79
5 Oct 11, 2025	SiriusAI-Text2SQL-32B <i>Tencent ML Platform</i>		32B	Many	72.43	74.40
6 May 07, 2025	Arctic-Text2SQL-R1-32B <i>Snowflake</i> <i>Snowflake AI Research</i>		32B	Few	72.20	73.84



Will programming languages be impacted by GenAI?

Nah, yeah!

Low-code

Low-code platforms are most often designed with developers in mind. Their primary goal is to accelerate the development process by providing a higher level of abstraction than traditional programming languages. While they offer extensive visual tooling, they are built on the assumption that a developer will need to extend the platform's capabilities with custom scripts, API integrations, and complex logic, giving them ultimate control over the final application.

No-code

No-code platforms typically cater to business users, citizen developers (sometimes referred to as *vibe coders*), and non-technical staff who have a deep understanding of business processes but may lack formal coding skills. These platforms are entirely declarative, meaning users build applications by defining what they want the app to do through menus and visual configurations, without writing any code at all. A prime example in the Google Cloud ecosystem is [AppSheet](#), which empowers users to build powerful applications directly from data sources like [Google Sheets](#) without any coding.

Source: <https://cloud.google.com/discover/what-is-low-code>



Will programming languages be impacted by GenAI?

Nah, yeah!

When to use low-code

Low-code platforms can excel in specific enterprise scenarios where speed, efficiency, and process automation are the primary drivers.

- **Internal tools:** Building custom admin panels, data entry applications, and internal dashboards can be ideal use cases, as the functional requirements are often standard and the need for delivery is immediate
- **Process automation:** Automating multi-step business workflows, such as employee onboarding or expense report approvals, can be accomplished quickly by visually connecting different systems and services
- **Rapid prototyping:** Developers can construct functional, interactive prototypes to validate application ideas and user flows with business stakeholders before committing to a full, code-intensive development cycle

When not to use low-code

Despite its advantages, low-code may not be the right solution for every development challenge. Recognizing its limitations is just as important as understanding its strengths.

- **Highly complex applications:** Systems requiring sophisticated algorithms, unique low-level performance optimizations, or unconventional computing architectures may be better suited for traditional, high-control coding
- **Mass-scale, highly customized public applications:** Building a consumer-facing social media platform or a global e-commerce site often requires granular control over every aspect of the user experience and infrastructure scaling, which can be constrained by a low-code platform
- **Deep, non-standard legacy integrations:** While low-code excels at connecting to modern APIs, integrating with proprietary, non-standard legacy systems that lack APIs may require complex custom development work that falls outside the scope of the platform

Source: <https://cloud.google.com/discover/what-is-low-code>



What is vibe coding?

Vibe coding is an emerging **software development practice that uses artificial intelligence (AI) to generate functional code from natural language prompts**, accelerating development, and making app building more accessible, **especially for those with limited programming experience.**

- **"Pure" vibe coding:** In its most exploratory form, **a user might fully trust the AI's output to work as intended.** As [Karpthy framed it](#), this is akin to "forgetting that the code even exists," making it **best suited for rapid ideation** or what he called **"throwaway weekend projects,"** where speed is the primary goal.
- **Responsible AI-assisted development:** This is the **practical and professional application of the concept.** In this model, AI tools act as a powerful collaborator or **"pair programmer."** The **user guides the AI but then reviews, tests, and understands the code it generates,** taking full ownership of the final product.



The Vibe Coding Process

Code-level workflow

1. **Describe the goal:** You start with a high-level prompt in plain language. For example: "Create a Python function that reads a CSV file."
2. **AI generates code:** The AI assistant interprets your request and produces the initial code.
3. **Execute and observe:** You run the generated code to see if it works as intended.
4. **Provide feedback and refine:** If the output isn't quite right or an error occurs, you provide new instructions, like, "That works, but add error handling for when the file is not found."
5. **Repeat:** This loop of describing, generating, testing, and refining continues until the code is complete.

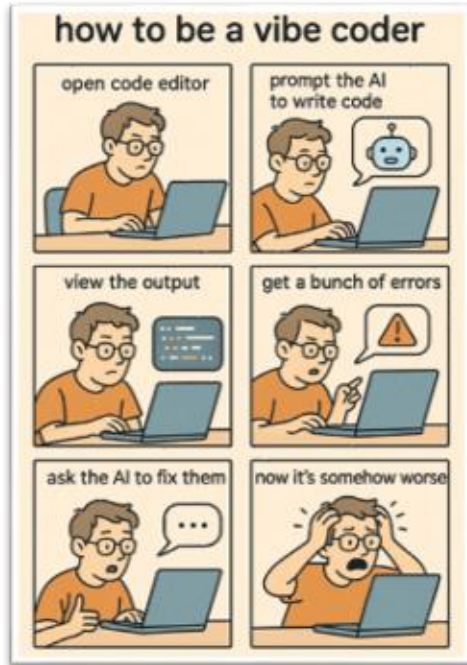
Application lifecycle

1. **Ideation:** You describe the entire application you want in a single, high-level prompt in tools like Google AI Studio or Firebase Studio.
2. **Generation:** The AI generates the initial version of the full application, including the UI, backend logic, and file structure.
3. **Iterative refinement:** You test the application and use follow-up prompts to add new features or change existing ones.
4. **Testing and validation:** A human expert reviews the application for security, quality, and correctness.
5. **Deployment:** With a final prompt or a single click, you deploy the application to a scalable platform like [Cloud Run](#).

<https://cloud.google.com/discover/what-is-vibe-coding>



What is vibe coding?



COMP6442 may cover this topic in more depth.

<https://cloud.google.com/discover/what-is-vibe-coding>

Feature	Traditional programming	Vibe coding
Code Creation	Manual coding line by line	AI-generated from natural language prompts
Developer or user role	Architect, implementer, debugger	Prompter, guide, tester, refiner
Coding expertise required	Higher (knowledge of programming languages and syntax)	Lower (understanding of the desired functionality)
Primary input	Precise code	Natural language prompts and feedback
Development speed	Generally slower, methodical	Potentially faster, particularly for prototyping simpler tasks
Error handling	Manual debugging based on code comprehension	Refinement through conversational feedback
Learning curve	Often steep	Potentially lower barrier to entry
Code maintainability	Relies on code quality, developer skill, and established practices	Can depend heavily on AI output quality and user review



BUGS



*“Program testing can be used to show the presence of bugs, but
never to show their absence”^[3]*

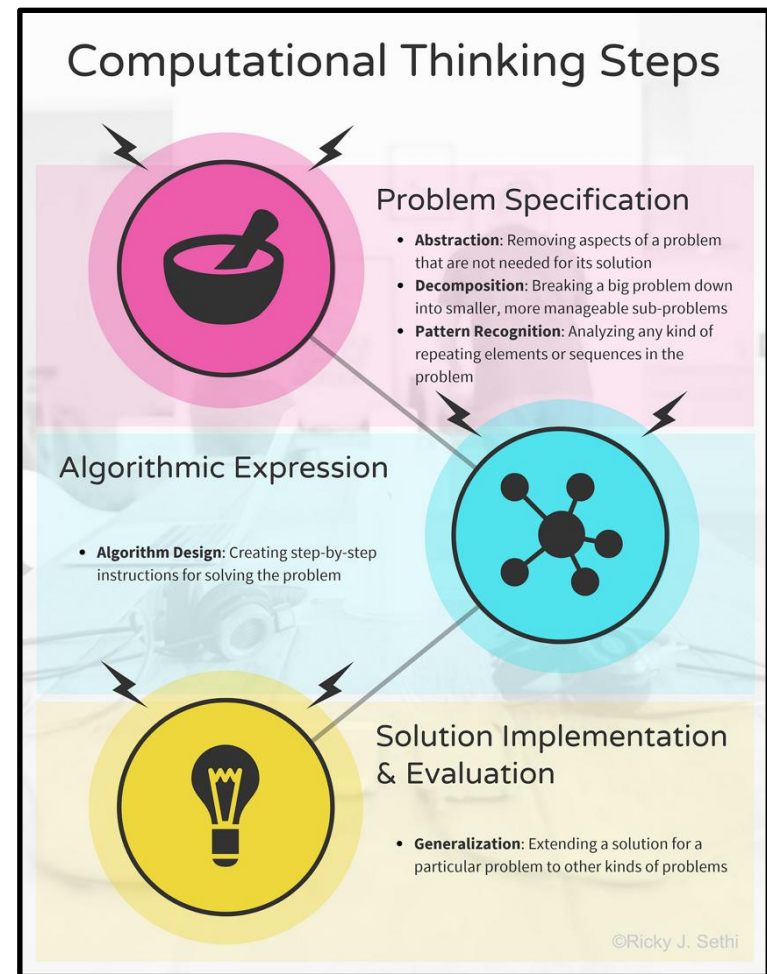
E. W. Dijkstra



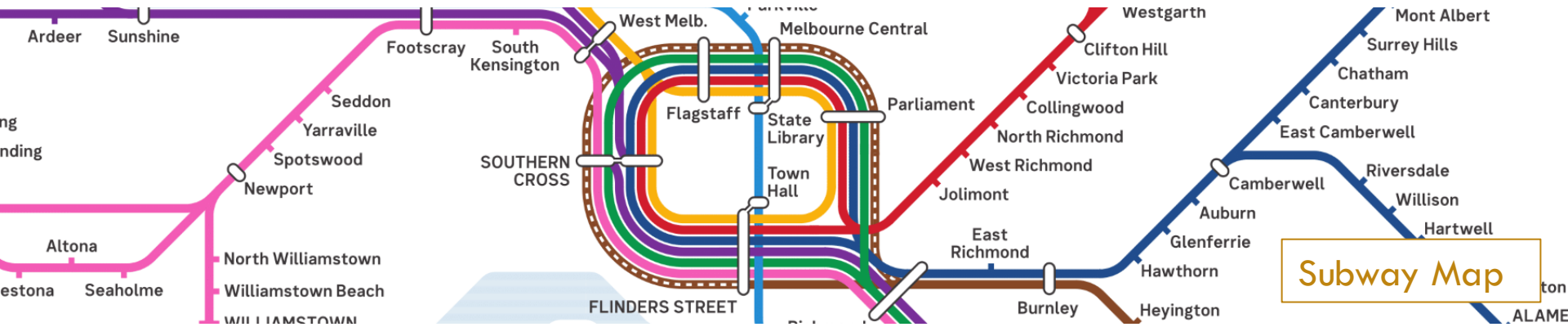
[3] E. W. Dijkstra, “Notes on Structured Programming,” Structured Programming, O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, Editors, Academic Press, London (1972), pp. 1-82.



BACK TO COMPUTATIONAL THINKING



ABSTRACTION – SIMPLIFYING COMPLEX SYSTEMS



Abstraction removes details on purpose

Abstraction is not about modelling the world accurately. It is about modelling the world enough for your goal.

What are we allowed to ignore?

>> What is relevant to solve the problem?

The metro map does not show geographical locations, actual distances, depths,... It only shows stations and trajectories.

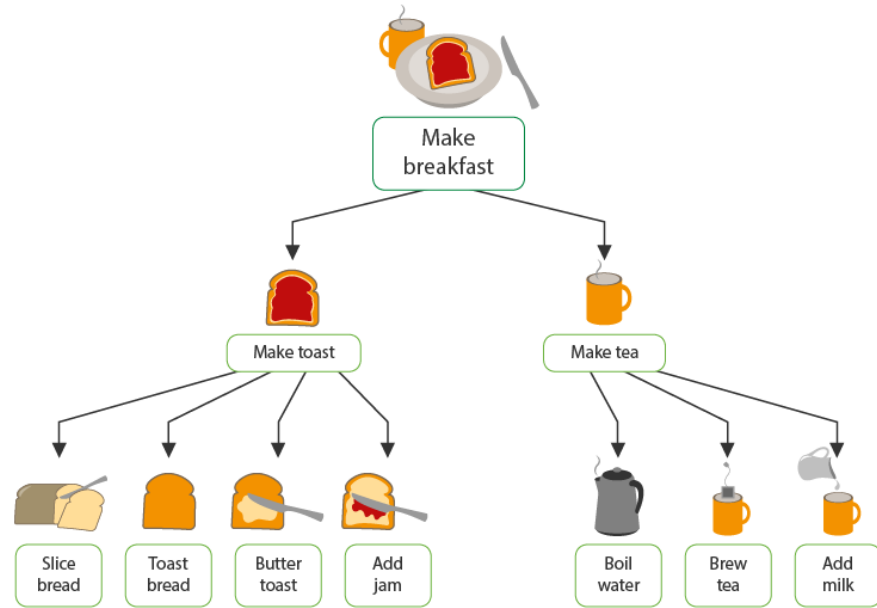


DECOMPOSITION

The process of **breaking down a complex problem / system into smaller, more manageable sub-problems**, making it easier to understand, analyse, and solve, often using a “divide and conquer” approach.

algorithm
design paradigm

Decompose a given problem into simpler subproblems. Examples: binary search, Euclidean algorithm (GCD), Merge Sort, etc.



PATTERN RECOGNITION

The process of finding **similarities** or **repeated structures across problems/data**, allowing us to **reuse logic, algorithms, or abstractions**.



Example: Cats share common characteristics (eyes, tail, fur, whiskers, ...) and behaviours (eat, meows, purrs, territory marking, ...).

These are patterns. We have a class of Cats that can derive specific cats (green eyes, grey fur, long tail).

Pattern recognition leads to classes, inheritance, and reuse.

Other examples: State Machines, Input-Process-Output, Bugs-Debugging Patterns.



ALGORITHM DESIGN

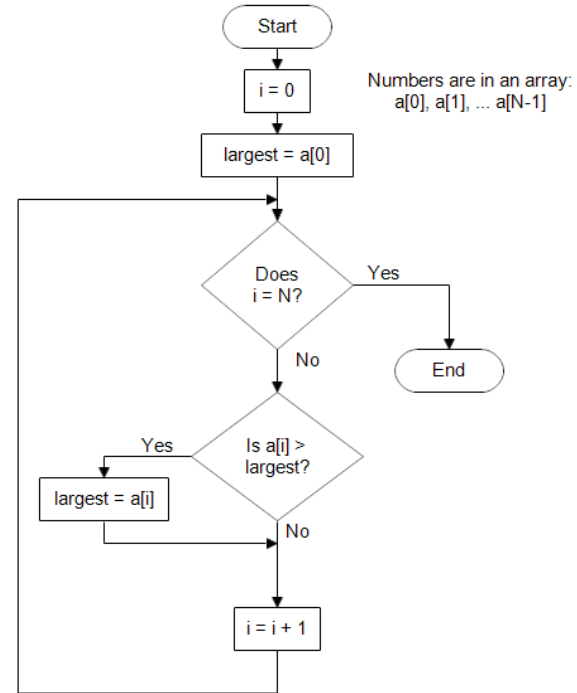
The process of designing a precise and unambiguous **sequence of steps** that a **computer can execute to solve a problem**.



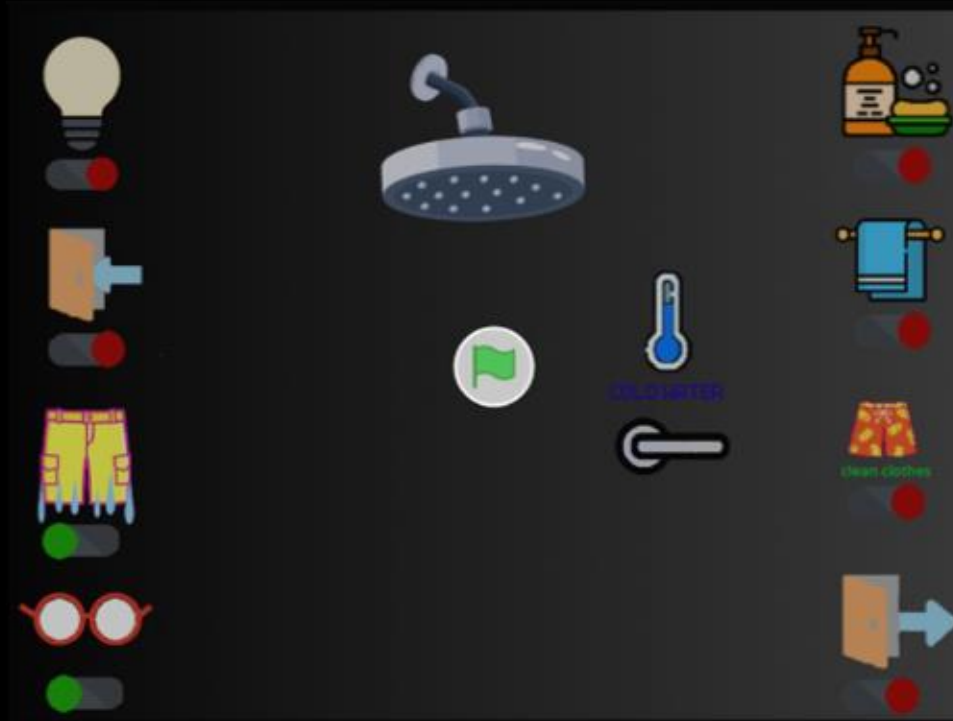
Examples of Design Strategies: divide-and-conquer, prune-and-search, dynamic programming, greedy strategy, randomization, linear programming,

[10, 20, 3, 40, 54, 32, 11, 23, 46, 81, 1]

Finding the Largest Number in a List of Numbers



THE SHOWER ALGORITHM



<EXERCISE TIME>

Consider the problem of designing a simple system to manage people in a university. The system must store basic information about students and teachers and support a small set of actions.

Using the principles of **Computational Thinking**, identify and explain:

- > how the problem can be decomposed,
- > what patterns can be recognised,
- > which aspects should be abstracted, and
- > how an algorithm could be designed to solve part of the problem.



Let's talk about JAVA

Java was first released in 1996, created by Sun Microsystems.

Designed for portability: “Write once, run anywhere”

More than a programming language, Java is a **platform**, including:

- > Rich **standard libraries** (networking, databases, graphics, etc.)
- > A **runtime environment (JVM)** that:
 - >> Runs Java programs on different machines
 - >> Manages **memory automatically** (garbage collection)
 - >> Provides **security and isolation**

Why Java is still widely used

- > Reliable and stable
- > Used in industry, research, and education
- > Strong tooling and ecosystem



A general-purpose, concurrent and high-level language that supports the object-oriented programming paradigm.



Let's talk about JAVA

1. Simple

> Simplified storage management, automatic garbage collection

>> considered complex in C/C++

> There are small versions of Java that can run in embedded devices

We wanted to build a system that could be programmed easily without a lot of esoteric training and which leveraged today's standard practice. So even though we found that C++ was unsuitable, we designed Java as closely to C++ as possible in order to make the system more comprehensible. Java omits many rarely used, poorly understood, confusing features of C++ that, in our experience, bring more grief than benefit.

Examples: No header files,
no pointer arithmetic/syntax

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

2. Object-Oriented (OO)

> “OO a programming paradigm based on objects – software entities that encapsulate data and function(s). An OOP computer program consists of objects that interact with one another.” (Wikipedia) [more details later in the course]

> Classes, Interfaces,... reusable software

🐼 The folks at Archimedes had lots of kinds of things in their simulation. Among them, ropes and elastic bands. In their initial C version of the product, they ended up with a pretty big system because they had to write separate software for describing ropes versus elastic bands. When they re-wrote their application in an object oriented style, they found they could define one basic object that represented the common aspects of ropes and elastic bands, and then ropes and elastic bands were defined as variations (subclasses) of the basic type. When it came time to add chains, it was a

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

3. Distributed

- > Designed from the start for networked environments (mid-1990s, rise of the Web)
- > Assumes programs communicate over networks, not just run locally
- > Early Java:
 - >> java.net (Sockets, URLs) included in the standard library
 - >> Java RMI (Remote Method Invocation)
 - >>> Call methods on remote objects as if they were local; Applets (code downloaded and executed from remote servers; object serialization)
- > Later versions introduced SOAP Web Services, RESTful services (still used today; resources has a URL; use HTTP methods (Get, Post, Delete), and more recently microservices/cloud (many small Java applications; the **Netflix case**: from monoliths to microservices)

COMP6331 covers networks in depth



<https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359> (Jan 2024)

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

4. Robust

> Java is designed to **prevent errors, detect failures early**, and keep programs **running safely even when things go wrong**

> *Emphasis on early checking for possible problems, later dynamic (runtime) checking, and on eliminating situations which are error prone (Gosling)*

> Strong type checking - Errors caught at compile time

> Automatic memory management (Garbage Collection) (no malloc/free, no pointer arithmetic/memory addresses can't be manipulated directly; null reference checks; Class cast checks; array bounds checking)

> Exception handling (controlled failure)

>> Failures are part of normal execution (by design)

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

5. Secure

> **It enables the construction of virus-free, tamper-free systems.**

> Before any Java program is executed, the Java Virtual Machine (JVM) performs bytecode verification (if it is structurally valid, type-safe, and compliant with the Java language rules).

>> *malformed or malicious bytecode is rejected before execution*

>> *Attackers cannot inject code that forges object references, corrupts the call stack, or executes unsafe instructions*

COMP6800 (and other courses) covers topics in security in more depth

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

6. Architecture-Neutral & 7. Portable

“Write once, run anywhere”

> The Java compiler **does not generate “machine code”** (native hardware instructions) → **it generates bytecodes**: a high-level, machine-independent code for a hypothetical machine that is implemented by the Java interpreter and run-time system. Java bytecodes are easily interpreted on any machine or dynamically translated into native machine code (for performance).

> It is not tied to a specific CPU, operating system, or hardware architecture (e.g., x86, ARM, 32-bit vs 64-bit). *Each platform (Windows, Linux, macOS, ARM, x86) has its own JVM implementation.*

> *The JVM translates bytecode into native instructions at runtime (via interpretation or JIT compilation)*

Java → Bytecode → JVM → Hardware

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic

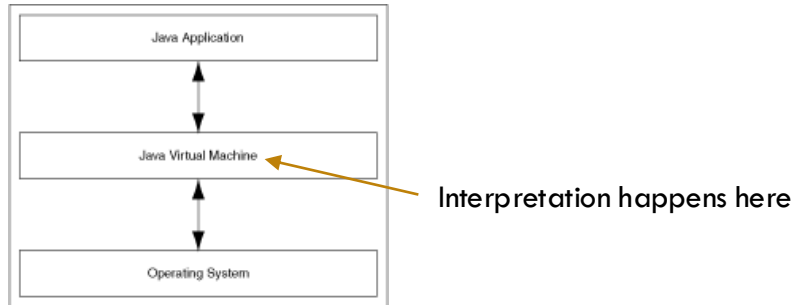


Let's talk about JAVA

8. Interpreted

> Java bytecode is not interpreted directly from source code like Python or JavaScript. Instead, **Java uses a two-stage execution model where bytecode is interpreted** (and or compiled) by the JVM.

> *Why is it called **bytecode**? “Java source compiles to low-level machine instructions. In Java, these instructions are known as bytecodes, because each instruction has a uniform size of one byte”. (Oracle, 19)*



More at: <https://docs.oracle.com/en/database/oracle/oracle-database/19/jjdev/Java-overview.html#GUID-5FE3D958-3D58-4976-B5E2-1B13F7B1BF47>

Source: J. Gosling, *Java: An overview*. Sun Microsystems, 1995.

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



Let's talk about JAVA

9. High-Performance

> The byte code can be translated on the fly (at runtime) into machine code for the particular CPU the application is running on.

10. Multithreaded

> *There are many things going on at the same time in the world around us.*
Multithreading allows a program to perform multiple operations concurrently

11. Dynamic

> Java classes are loaded on demand at runtime by the JVM (final structure of the running program is not known at compile time); Late (dynamic) binding via polymorphism; Runtime linking and verification,...

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



<EXERCISE TIME>

For each scenario below, select 3-5 concepts that matter the most.

- > Explain why each concept matters.
- > Explain which Java features support each concept

GenAI can help you with this...

Scenario A – Banking System

- . Millions of users
- . Sensitive financial data
- . Must run for years without crashing

Scenario B – Multiplayer Game

- . Thousands of concurrent players
- . Real-time interactions
- . Needs fast response

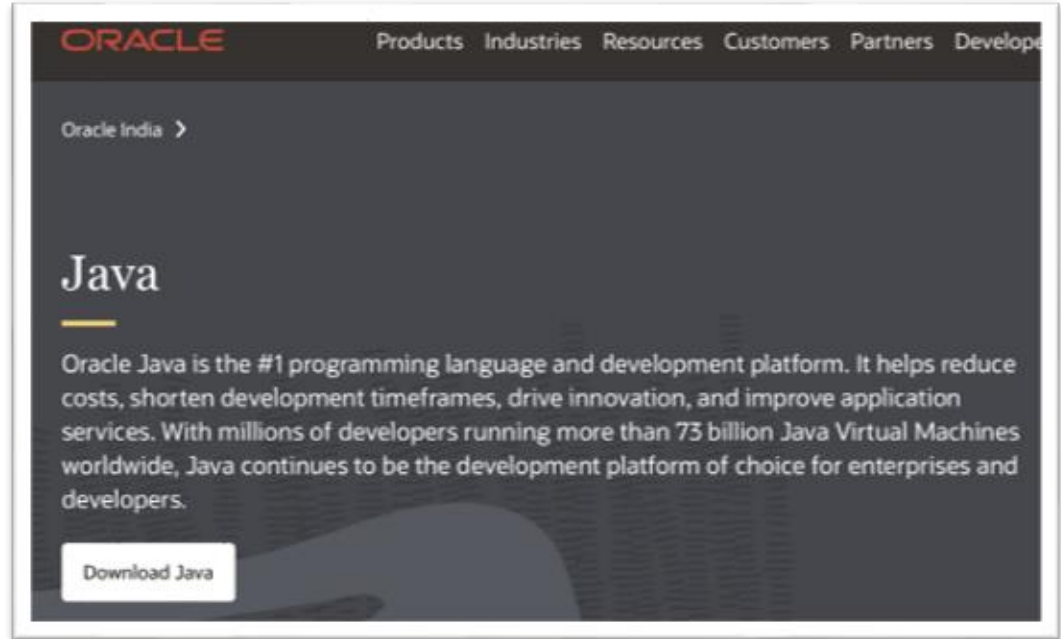
Scenario C – Uni Learning Platform

- . Runs on students' laptops (Windows, macOS, Linux)
- . Frequent updates during semester
- . Third-party plugins

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-Neutral
7. Portable
8. Interpreted
9. High-Performance
10. Multithreaded
11. Dynamic



MEME



Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)

CRICOS PROVIDER CODE: 00120C